

mmdh - ex06

- 1.1. In nutshell the goal of the baseline process is to compress an image into smaller size while losing minimal (acceptable) amount of detail
- 1.2. Image Scale + Blocks Formation = Pad to multiple of MCU size ; divide into blocks (lossless)
Color Space Conversion = Separate luma from chroma (lossless)
Subsampling = Reduce chroma resolution (exploiting human insensitivity to color detail (lossy))
Quantization = Discard insignificant high-frequency coefficients (lossy)
Forward DCT = Spatial $\rightarrow$ Frequency domain (lossless)
Entropy coding = lossless compression by reducing statistical redundancy (lossless)
- 2.1. Because human vision is more sensitive to brightness than color, by splitting (brightness) from YCbCr, we can subsample color channel aggressively and lose almost no perceived quality, while cutting $\frac{1}{3}$ to $\frac{1}{2}$ of the data
- 2.2. 4:4:4 $\rightarrow$ Full range RGB, most color accuracy but also most storage needed
4:2:2 $\rightarrow$ Reduced chroma (half), mild softening of color edgess
4:2:0 $\rightarrow$ further reduced chroma (quarter)
- 2.3. The forward DCT operates in a 8×8 block, so both image width and height must be divisible by 8.
- 3.1.
 - Level - Shift pixel values by by subtracting $2^{(\text{bit depth} - 1)}$
 - Divide image into 8×8 pixel blocks
 - then apply forward DCT to each blocks thus represents horizontal/vertical frequencies from DCF-DCT
- 3.2. High frequency coefficients first because it could be quantized more aggressively thus achieving (lossy) compression
- 3.3. DC Coefficient = DCT(0,0), Large DC value average brightness of the 8×8 block
AC Coefficient = the other 63 value - describe how Frequency changes
- 4.1. Advantages :
 - Saves data
 - Aggressively deletes high-frequency details that the eyes can't see anyway
- disadvantages :
 - quantization error is irreversible
 - Can cause blockiness at high compression
- What to keep in mind: the human eyes is sensitive to low frequencies so so the quantization for low frequency entries should be

Kept small while we can have high quantization for high frequency entries.

4.2. Larger quantization results to a smaller file size but also may introduce more artifacts. High enough Q may be "visually lossless" but low enough Q may start introduce obvious blocky artifacts.

5.1. It is a lossless process by removing only the redundant way of expressing colors by encoding repeated color pattern. For example A,A,A,A,A to '5XA'

5.2. RLE compresses multiple runs of the same pattern instead instantly jumps to the next pattern

5.3. Because the DC Coefficients represents "brightness" which is what the human eyes are more sensitive of. DC-encoding is generally for DCT, while AC-coefficients are used for RLE and Huffman coding

6. Encoding { T A T T A R R A T T A T }

Initial dictionary { T:0, A:1, R:2 }, next code = 3
+1

12 Characters, 12 Steps

Step	Char	Current String(w)	WtC in dict	Action	Output	Add to dict	Dictionary
1	T	" "	—	w = "T"	—	—	{T:0, A:1, R:2}
2	A	"T"	"TA" not in dict	w = "A"	0	TA = 3	+TA = 3
3	T	"AT"	"AT" not in dict	w = "T"	1	AT = 4	+AT = 4
4	T	"TT"	"TT" not in dict	w = "T"	0	TT = 5	+TT = 5
5	A	"T"	"TA" in dict	w = "TA"	—	—	no change
6	R	"TA"	"TAR" not in dict	w = "R"	3	TAR = 6	+TAR = 6
7	R	"RR"	"RR" not in dict	w = "R"	2	RR = 7	+RR = 7
8	A	"R"	"RA" not in dict	w = "A"	2	RA = 8	+RA = 8
9	T	"A"	"AT" in dict	w = "AT"	—	—	no change
10	T	"AT"	"ATT" not in dict	w = "T"	4	ATT = 9	+ATT = 9
11	A	"T"	"TA" in dict	w = "TA"	—	—	no change
12	T	"TA"	"TAT" not in dict	w = "T"	3	TAT = 10	+TAT = 10
end	—	"T"	—	out = 0	0	—	—

Encoded output [0, 1, 0, 3, 2, 2, 4, 3, 0] (9 codes)

Original = $12 \times 8 \text{ bits} = 96 \text{ bits}$
 Encoded = $9 \times 4 \text{ bits} = 36 \text{ bits}$
 Compression = $2.7 \times$

Decoding $[0, 1, 0, 3, 2, 2, 4, 3, 0]$
 Initial dict: $[0: T, 1: A, 2: R]$
 Next code: 3

Step	Code	Decode to	output	Add to dict	Dictionary
1	0	T	T	-	$\{0: T, 1: A, 2: R\}$
2	1	A	A	$T+A: 3$	$+TA = 3$
3	0	T	T	$A+T: 4$	$+AT = 4$
4	3	TA	TA	$T+T: 5$	$+TT = 5$
5	2	R	R	$TA+R: 6$	$+TAR = 6$
6	2	R	R	$R+R: 7$	$+RR = 7$
7	4	AT	AT	$R+A: 8$	$+RA = 8$
8	3	TA	TA	$AT+T: 9$	$+ATT = 9$
9	0	T	T	$TA+T: 10$	$+TAT = 10$

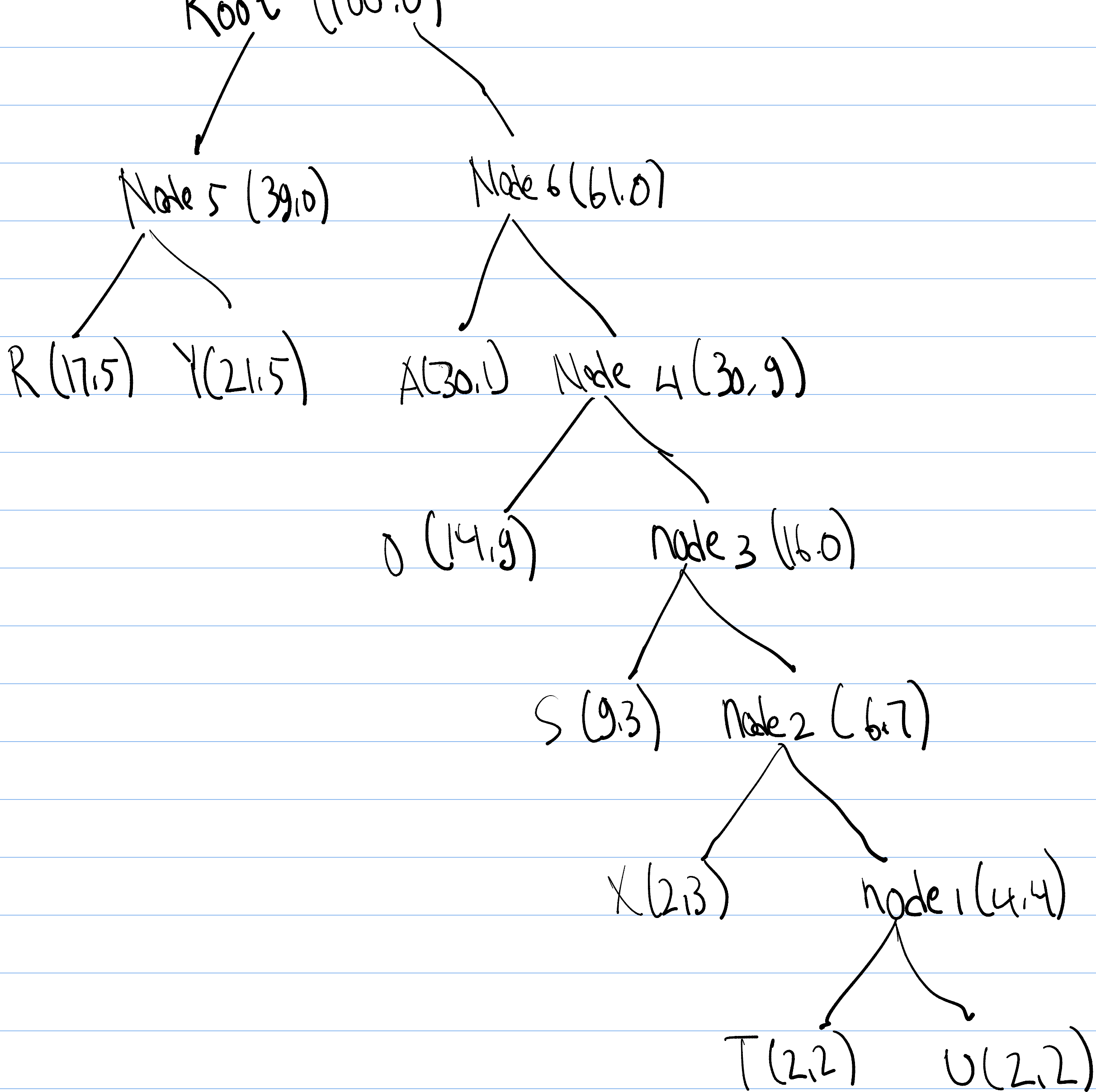
Decoded = T A T T A R R A T T A T

7 Sort by probability (Combine smallest pairs)

T	2.2%
U	2.2%
X	2.3%
S	9.3%
O	14.9%
R	17.5%
Y	21.5%
A	30.1%

Combine T(2.2) + U(2.2) = node 1 (4.4)
 node 1(4.4) + X(2.3) = node 2 (6.7)
 node 2(6.7) + S(9.3) = node 3 (16.0)
 node 3(16.0) + O(14.9) = node 4 (30.9)
 Combine R(17.5) + Y(21.5) = node 5 (39.0)
 Node 4 (30.9) + A(30.1) = node 6 (61.0)
 Combine node 5 (39.0) + node 6 (61.0) = 100

D. L (100.0)



Huffman Codes (0 = left, 1 = Right)

R = 00 2 bits

Y = 01 2 bits

A = 10 2 bits

O = 110 3 bits

S = 1110 4 bits

X = 11110 5 bits

T = 111110 6 bits

U = 111111 6 bits

Avg = 2.581 bits / symbol